



Uncovering the Linkage between Healthy Foods and their Nutrients

CIS5450 Final Project

By: Max Eldabbas, Anthony Lam, and Yechezkel (Chezky) Rothman

TA: Shailesh Sridhar

Introduction /Background

Our Project Plan

The dataset we chose to base our study on was called “*Open Food Facts*,” which consists of nutrition facts from foods around the world.



- We studied the **correlation between the ingredients in food to how healthy they are** in terms of fats, oils, vitamins, fibers, etc.
- The **ultimate objective** of this project was to find if there is a correlation between certain nutrients and the overall custom health score we calculate. Another goal of this project was to discover if there was a linkage or correlation to overall carbon footprint, however, the carbon data was 99% null values.
- **Types of models** that we tested:
 - i. Random Forest Classification
 - ii. SVM
 - iii. Logistic Regression.

Why is this project interesting?

This project is interesting because it combines nutrition, data science, and sustainability to provide valuable insights into our food choices. This project can give insight into the health impacts of different foods and their components, helping consumers make smart decisions in regards to the food they choose to buy. Additionally, if we would be able to integrate carbon footprint analysis, this would add an environmental perspective, making our findings useful for consumers, researchers, and policymakers alike.

Data Preprocessing, Exploration, Cleaning, Feature Engineering



First Look at the Data

At first glance, this dataset immediately seemed challenging to deal with:

- Several unnecessary columns (to our study)
- Many NaN values
- Some values are in different languages

	code	url	creator	created_t	created_datetime	last_modified_t	last_modified_datetime	product_name	generic_name	quantity	...	fruits-vegetables-nuts_100g	fruits-vegetables-nuts-estimate_100g	coring-meat-protein-ratio_100g	cocoa_100g	chlorophyll_100g	carbon-footprint_100g	nutrition-score_fr_100g	nutrition-score_uk_100g	glycemic-index_100g	water-hardness_100g
0	3087	http://world-en.openfoodfacts.org/product/0000...	openfoodfacts-contributors	1474103866	2016-09-17T09:17:46Z	1474103893	2016-09-17T09:18:13Z	Farine de blé noir	NaN	1kg	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	4530	http://world-en.openfoodfacts.org/product/0000...	usda-ndb-import	1489069957	2017-03-09T14:32:37Z	1489069957	2017-03-09T14:32:37Z	Banana Chips Sweetened (Whole)	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	14.0	14.0	NaN	NaN
2	4559	http://world-en.openfoodfacts.org/product/0000...	usda-ndb-import	1489069957	2017-03-09T14:32:37Z	1489069957	2017-03-09T14:32:37Z	Peanuts	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	0.0	0.0	NaN	NaN
3	16087	http://world-en.openfoodfacts.org/product/0000...	usda-ndb-import	1489055731	2017-03-09T10:35:31Z	1489055731	2017-03-09T10:35:31Z	Organic Salted Nut Mix	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	12.0	12.0	NaN	NaN
4	16094	http://world-en.openfoodfacts.org/product/0000...	usda-ndb-import	1489055653	2017-03-09T10:34:13Z	1489055653	2017-03-09T10:34:13Z	Organic Polenta	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5 rows × 163 columns																					

Data Cleaning (Preprocessing)

This dataset was trickier to work with than we originally imagined. There are a total of 163 columns in this dataset, of which only 34 of these columns contained data where at least 50% were non-null.

We combated this issue as well as generally cleaned the data by performing a series of **pre-processing** steps:

- Dropping columns that we deemed unnecessary
- Replacing missing values for all columns to be consistent (i.e., object values filled with 'Unknown' and integer columns filled '0.')
- Dropping duplicate rows

```
[ ] 1 # Checking for missing values, inspect data types, get summary, visualize
    2 print("SHAPE:")
    3 print(open_food_facts.shape)
    4 print()
    5 print("DTYPES:")
    6 print(open_food_facts.info())
    7 print()
    8 print("MISSING VALUES:")
    9 print(open_food_facts.isnull().sum())
    10
    11 missing_50 = (open_food_facts.isnull().mean() > 0.5).sum()
    12 print(f"Number of columns with more than 50% missing values: (missing_50)")
    13
    14 print()
    15 print("SUMMARY:")
    16 print(open_food_facts.describe())
```

SHAPE:
(356827, 163)

DTYPES:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 356827 entries, 0 to 356826
Columns: 163 entries, code to water-hardness_100g
dtypes: float64(187), object(56)
memory usage: 442.8+ MB
None

MISSING VALUES:

code	26
url	26
creator	3
created_t	3
created_datetime	10
...	...
carbon-footprint_100g	355749
nutrition-score-fr_100g	101171
nutrition-score-uk_100g	101171
glycemic-index_100g	356827
water-hardness_100g	356827
Length: 163, dtype: int64	
Number of columns with more than 50% missing values: 129	

Dealing with missing values

Since we wanted the most accurate models that we could build, we decided to drop all columns that had more than 75% of their values missing:

```
open_food_facts = open_food_facts.dropna(thresh=len(open_food_facts) * 0.25, axis=1)
```

This resulted in 39 columns of data left at our disposal. Fortunately, these were all columns that we were most interested in studying.

The DataSet

Some quite notable columns from this dataset that we used a combination of these metrics to determine a health_score, which was calculated by dividing the sum of unhealthy nutrients by the sum of healthy ingredients in each type of food listed within the dataset. Here was our calculation:

- fat_100g
- proteins_100g
- fiber_100g
- vitamin-a_100g
- iron_100g
- Saturated-fat_100g
- ingredients_from_palm_oil_n

```
open food facts ['health score'] = (open food facts ['proteins 100g'] +  
open food facts ["fiber 100g"] + open food facts ["vitamin-a 100g"] +  
open food facts ["vitamin-c 100g"] + open food facts ["calcium 100g"] +  
open food facts ["iron 100g"]) / (open food facts ["saturated-fat 100g"] +  
open food facts ["trans-fat 100g"] + open food facts ["sugars 100g"] +  
open food facts ["sodium 100g"]).replace(0, np.nan)
```

Feature Engineering

Beyond the health score calculation, we also decided to scale all health scores such that their scores were out of 100 (rather than a seemingly random value).

- First, we performed a log transformation of the `health_score` data, which was essential to reduce the impact of large/skewed values.
- Next, we created a new `health_score_scaled` column that was made by normalizing the log transformed health scores to a range between 0 and 100.

```
open_food_facts['health_score_log'] =  
np.log1p(open_food_facts['health_score'])
```

```
open_food_facts['health_score_scaled'] =  
(open_food_facts['health_score_log'] -  
open_food_facts['health_score_log'].min()) /  
(open_food_facts['health_score_log'].max() -  
open_food_facts['health_score_log'].min()) * 100
```

health_score	health_score_log	health_score_scaled
0.167818	0.155137	4.123288
1.382236	0.868039	7.162253
2.667673	1.299557	9.001731
1.575146	0.945906	7.494183
1.155228	0.767897	6.735363

Feature Engineering cont.

▼ Feature Engineering

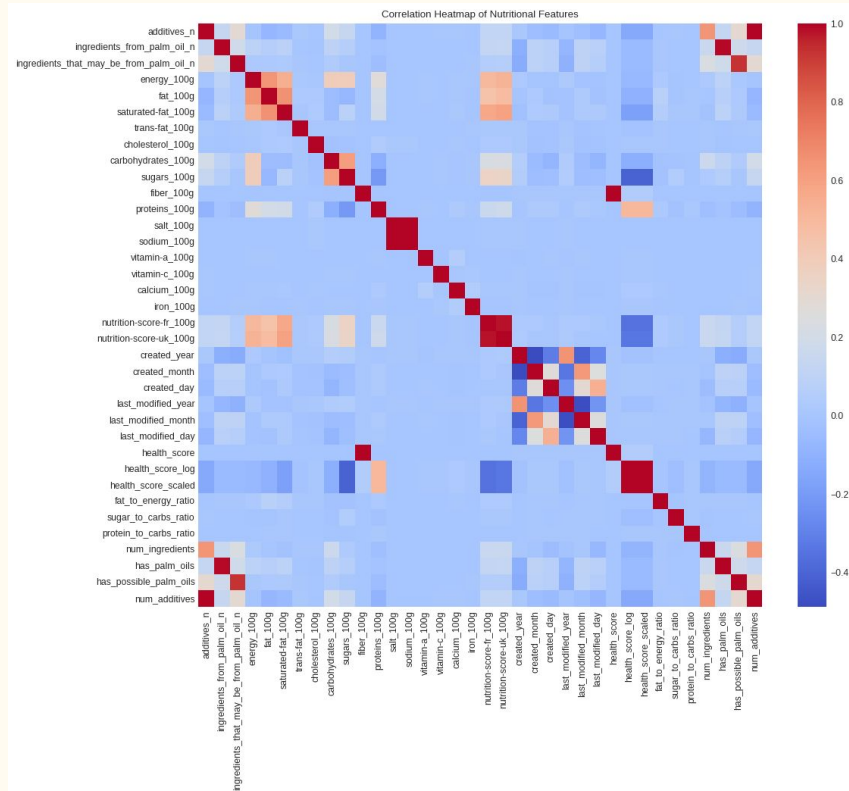
```
[70] 1 # Feature Engineering
2
3 # Date time features
4 open_food_facts['created_year'] = open_food_facts['created_datetime'].dt.year
5 open_food_facts['created_month'] = open_food_facts['created_datetime'].dt.month
6 open_food_facts['created_day'] = open_food_facts['created_datetime'].dt.day
7 open_food_facts['last_modified_year'] = open_food_facts['last_modified_datetime'].dt.year
8 open_food_facts['last_modified_month'] = open_food_facts['last_modified_datetime'].dt.month
9 open_food_facts['last_modified_day'] = open_food_facts['last_modified_datetime'].dt.day
10
11 # Custom health score feature calculated by dividing the sum of unhealthy nutrients by the sum of healthy ingredients
12 open_food_facts['health_score'] = (open_food_facts['proteins_100g'] + open_food_facts["fiber_100g"] + open_food_facts["vitamin-a_100g"] + open_food_facts["vitamin-c_100g"] + open_food_facts["calcium_100g"] + open_food_facts["iron_100g"]) / (open_food_facts["sugars_100g"] + open_food_facts["total_fat_100g"] + open_food_facts["sodium_100g"] + open_food_facts["total_saturated_fat_100g"] + open_food_facts["total_cholesterol_100g"] + open_food_facts["total_sugar_100g"] + open_food_facts["total_fiber_100g"] + open_food_facts["total_vitamin_a_100g"] + open_food_facts["total_vitamin_c_100g"] + open_food_facts["total_calcium_100g"] + open_food_facts["total_iron_100g"])
13
14 # Scale the health score to be out of 100 (while also cutting out the outliers)
15 open_food_facts['health_score_log'] = np.log1p(open_food_facts['health_score'])
16 open_food_facts['health_score_scaled'] = (open_food_facts['health_score_log'] - open_food_facts['health_score_log'].min()) / (open_food_facts['health_score_log'].max() - open_food_facts['health_score_log'].min()) * 100
17
18 # open_food_facts['health_score'] = open_food_facts['health_score'].fillna(0)
19 # open_food_facts['health_score_scaled'] = open_food_facts['health_score_scaled'].fillna(0)
20 open_food_facts = open_food_facts.dropna(subset=['health_score_scaled'])
21
22 # Custom nutrient ratios
23 open_food_facts['fat_to_energy_ratio'] = open_food_facts['fat_100g'] / open_food_facts['energy_100g'].replace(0, np.nan)
24 open_food_facts['sugar_to_carbs_ratio'] = open_food_facts['sugars_100g'] / open_food_facts['carbohydrates_100g'].replace(0, np.nan)
25 open_food_facts['protein_to_carbs_ratio'] = open_food_facts['proteins_100g'] / open_food_facts['carbohydrates_100g'].replace(0, np.nan)
26
27 # Find number of ingredients in each product
28 open_food_facts['num_ingredients'] = open_food_facts['ingredients_text'].str.count(',') + 1
29
30 # See if there is palm oil in the product
31 open_food_facts['has_palm_oils'] = (open_food_facts['ingredients_from_palm_oil_n'] > 0).astype(int)
32 open_food_facts['has_possible_palm_oils'] = (open_food_facts['ingredients_that_may_be_from_palm_oil_n'] > 0).astype(int)
33
34 # Find number of additives
35 open_food_facts['num_additives'] = open_food_facts['additives_n'].fillna(0).astype(int)
36
37 open_food_facts.head(5)
```

Correlation Heatmap of Nutritional Features

Upon first glance, we can see that there are a few features that have a strong positive correlation, such as:

- ‘Fat_100g’ and ‘energy_100g’
- ‘Fat_100g’ and ‘saturated-fat_100g’

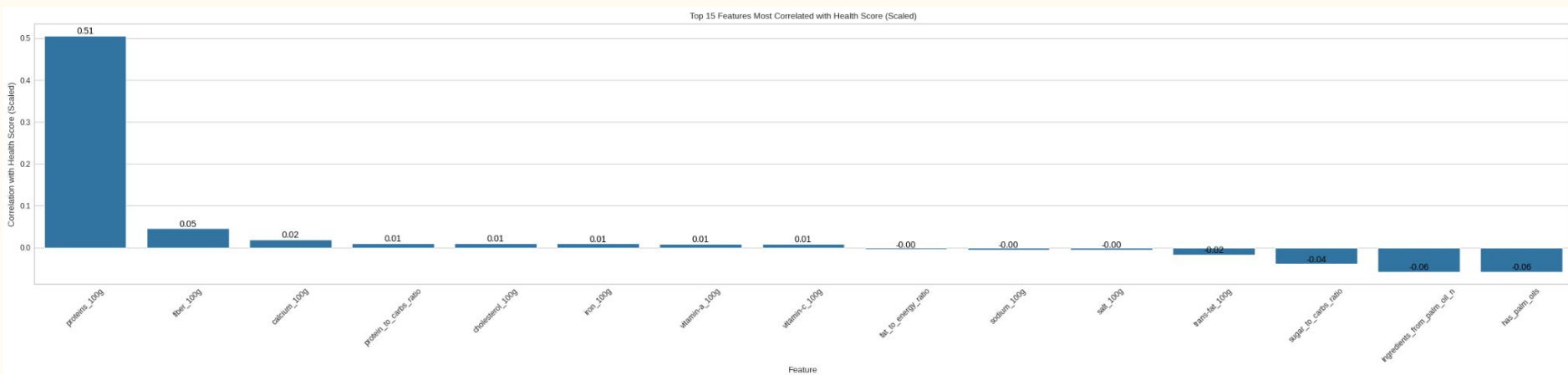
An interesting negative correlation to point out is ‘sugars_100g’ vs our custom ‘health_score_scaled’ feature. This makes intuitive sense because the higher the amount of sugar, the lower the health score.



Top 15 Features Most Correlated w/ Health Score

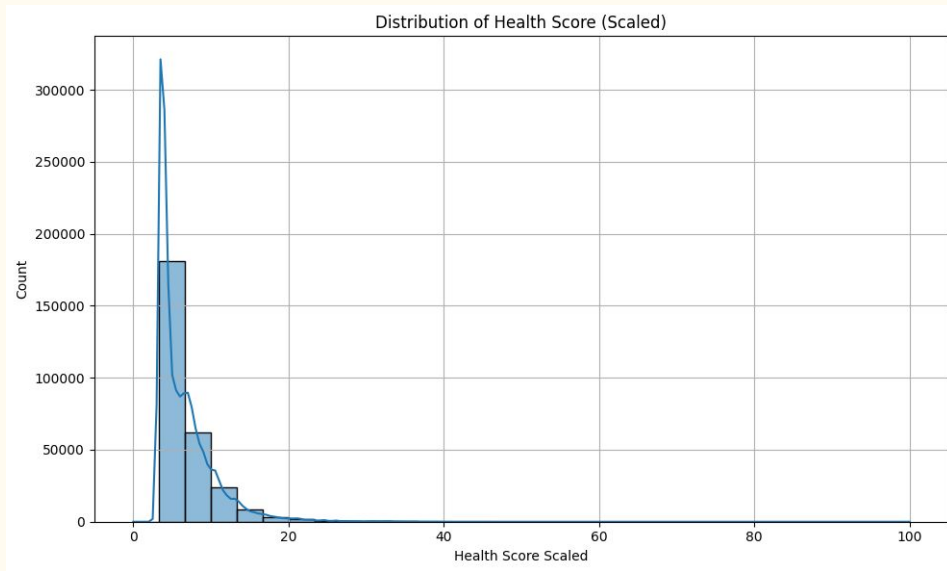
Some key insights to show here is that proteins_100g has a very strong positive correlation of 0.51 with the health score, which suggests that foods with a higher protein content makes up for most of the health score. Then follows fiber and calcium which have weaker correlations of about 0.05 and 0.02, respectively.

We can also find that the correlation decreases with proven unhealthy ingredients, such as sugars and palm oils.



Taking a Look at Raw Health Scores

When examining the scaled health scores (out of 100), it seems that most of our scores are skewed heavily below a score of 20. The problem in using these scores as is, is that any model trained on this data will most likely underfit the higher scores and will place all scores in a lower score region. Our next aim is to balance out these scores for better distribution and therefore maximize our model generalization.



Turning Health Score (scaled) into a Classification Column

```
[ ] 1 # Binning health score classes by quantile so that all classes of health scores are relative to each other
    2 open_food_facts['health_score_class'] = pd.qcut(open_food_facts['health_score_scaled'], q=3, labels=[0, 1, 2])

[ ] 1 features = ['num_ingredients', 'fat_to_energy_ratio', 'sugar_to_carbs_ratio', 'protein_to_carbs_ratio', 'num_additives', 'has_palm_oils', 'has_possible_palm_oils', 'created_year', 'created_month', 'created_day', 'last_modified_year', 'last_modified_month', 'last_modified_day']
    2
    3 X = open_food_facts[features]
    4 y = open_food_facts['health_score_class']
    5
    6 seed = 42
    7
    8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state = seed)

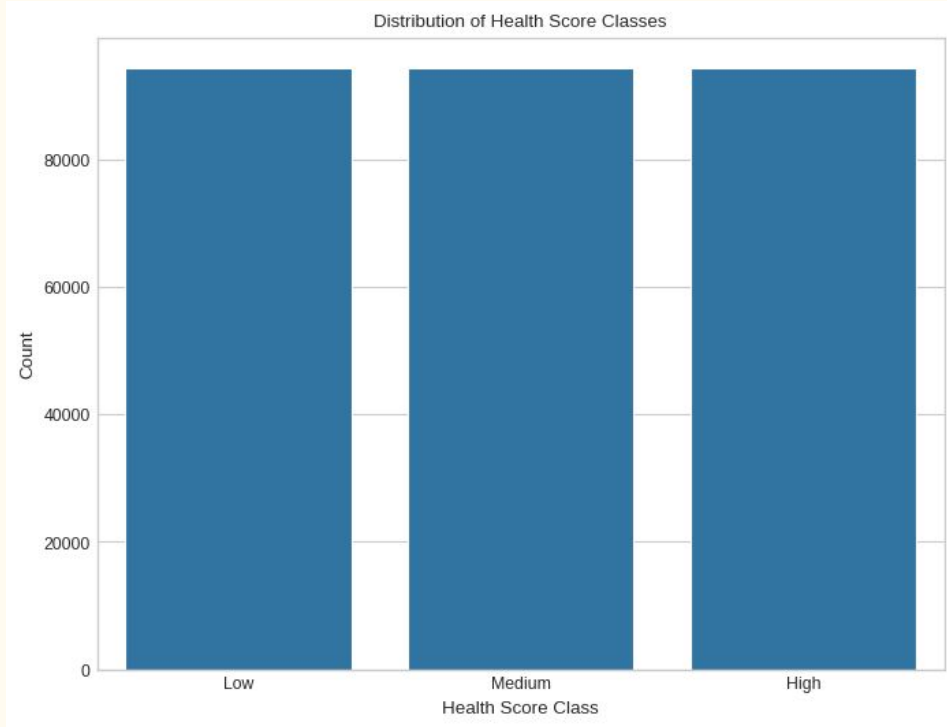
[ ] 1 model = RandomForestClassifier(class_weight='balanced', n_estimators = 120, max_depth = 20, random_state = seed)
    2 model.fit(X_train, y_train)
    3
    4 y_pred = model.predict(X_test)
    5
    6 r2 = r2_score(y_test, y_pred)
    7 mse = mean_squared_error(y_test, y_pred)
    8
    9 print(classification_report(y_test, y_pred))
   10
   11 confusion = confusion_matrix(y_test, y_pred)
   12 labels = ['Low', 'Medium', 'High']
   13 plt.figure(figsize=(8, 6))
   14 sns.heatmap(confusion, annot=True, fmt='d', cmap='Blues', xticklabels = labels, yticklabels = labels)
   15
   16 plt.title('Confusion Matrix')
   17 plt.xlabel('Predicted')
   18 plt.ylabel('Actual')
   19 plt.show()
```

Here, we divided the `health_score_scaled` column into three quantile-based bins, which separates them into a low, medium and high health score to balance out these classes. We named this feature `'health_score_class.'` Next, we define our features and target feature and use these to split our dataset. We then use a random forest model with a balanced `class_weight` to handle any imbalances. We will view the results in the next slides.

Model's Distribution of Classes

From the resultant quantile-based binning, we see that each health score class now has a nearly identical amount of samples that we can now use for our modeling.

Now that we have a balanced set of health scores, we are ready to train our models.



Health Score Turned Classification Problem

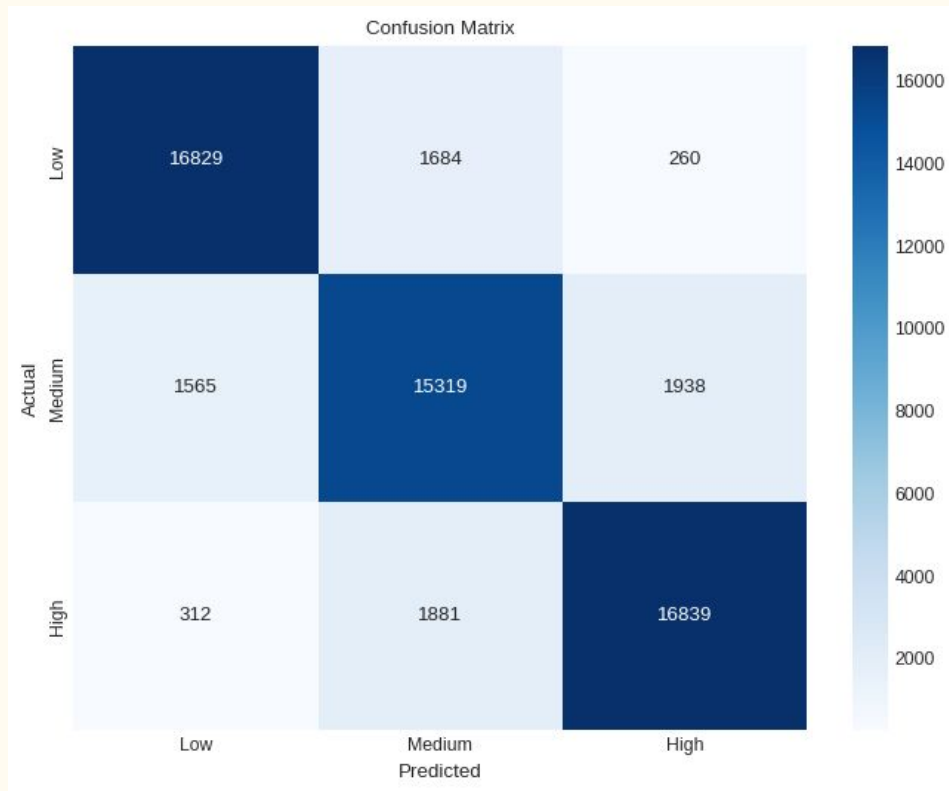
We went on to first create a Random Forest Classifier model based on a 'balanced' class weight, 120 estimators, and a random state of 42.

```
model = RandomForestClassifier(class_weight='balanced', n_estimators = 120, max_depth = 20, random_state = seed)
```

Judging by the classification report generated, our model is 87% accurate. From the confusion matrix, it predicts health-low foods correctly 16829 times, health-medium foods 15319 times, and health-high foods correctly 16839 times.

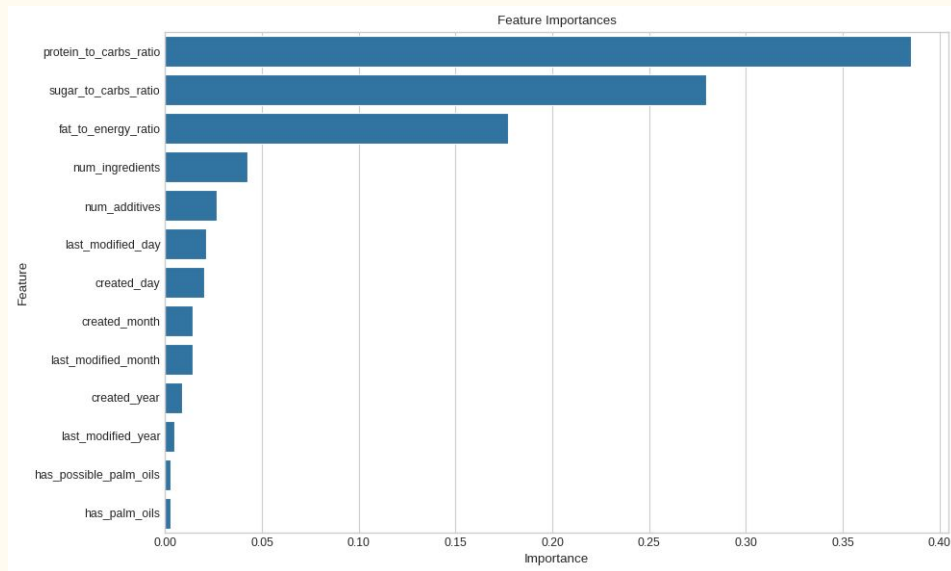
The classification report shows that our model gets 87% of the predictions correctly overall and treats all classes equally (the macro avg).

	precision	recall	f1-score	support
0	0.90	0.90	0.90	18773
1	0.81	0.81	0.81	18822
2	0.88	0.88	0.88	19032
accuracy			0.87	56627
macro avg	0.87	0.87	0.87	56627
weighted avg	0.87	0.87	0.87	56627



Feature Importance

The top 3 features are ratios between different health factors, all of which are known to contribute positively or negatively to overall health. Here we see that protein to carbs and sugar to carbs ratios play a major role in determining the health score of a food item.



Data Splitting, Regression Modeling, Model Evaluation

Modeling

Uncovering different types of models.

- Random Forest Classification
- Logistic Regression
- SVM



Logistic Regression

Performance: Moderate performance with an accuracy of 58.65%, precision of 57.54%, recall of 58.65%, and F1-score of 56.80%. The lower F1-score suggests some imbalance in precision and recall, possibly due to misclassification in certain classes.

Strengths: Logistic Regression is computationally efficient and highly interpretable, with coefficients directly indicating feature impacts on class probabilities.

Weaknesses: The model struggles with the dataset's complexity, as nutritional features will likely have non-linear relationships that Logistic Regression cannot capture effectively. The lower performance indicates that the data is not well-separated by linear boundaries.

```
# Model #1 Logistic Regression
logistic_regression_model = LogisticRegression(
    max_iter = 1000,
    class_weight = 'balanced',
    random_state = seed
)
```

Random Forest Classification

Performance: Achieves the highest accuracy (86.20%), precision (86.19%), recall (86.20%), and F1-score (86.19%), indicating a balanced and robust performance across all classes. The near-identical metrics suggest the model effectively handles class distribution.

Strengths: Captures non-linear relationships and feature interactions due to its grouping of decision trees.

Weaknesses: Computationally intensive due to multiple trees, leading to longer training times compared to linear models. Interpretability is moderate, relying on feature importances rather than direct coefficient analysis.

```
# Model #2 Random Forest
random_forest_model = RandomForestClassifier(
    n_estimators = 120,
    max_depth = 20,
    random_state = seed,
    class_weight = 'balanced'
)
```

Linear SVM

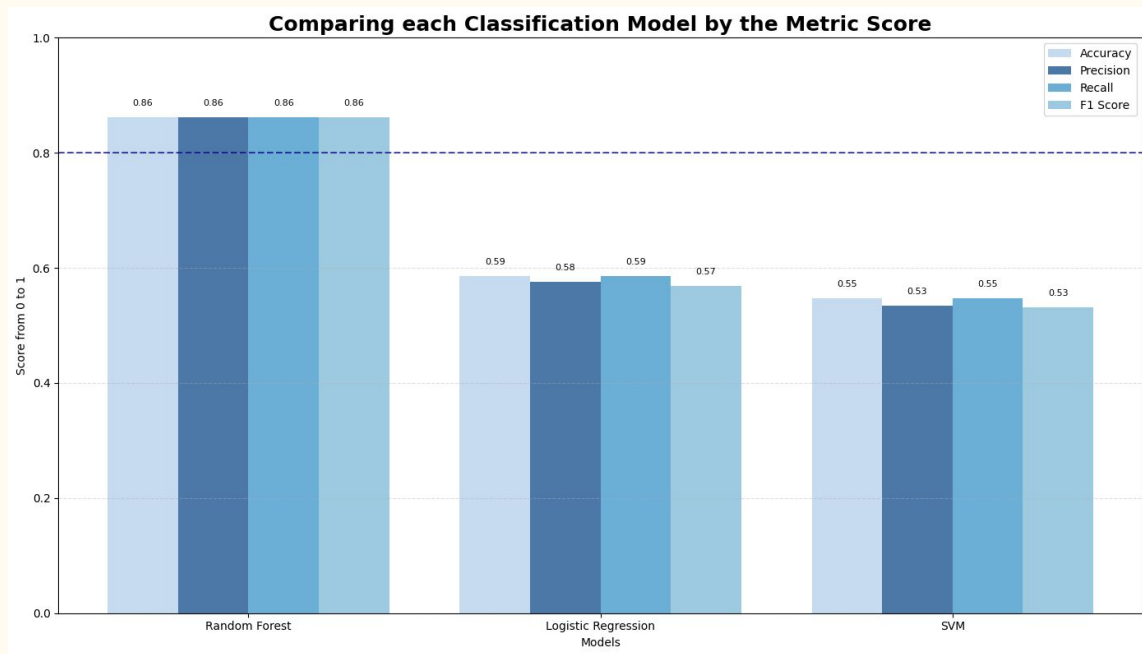
Performance: The lowest performance, with an accuracy of 54.69%, precision of 53.39%, recall of 54.69%, and F1-score of 53.20%. Similar to Logistic Regression, the F1-score is slightly lower, indicating challenges in balancing precision and recall.

Strengths: Linear SVM is efficient for high-dimensional data and can handle large datasets with clear linear margins.

Weaknesses: Like Logistic Regression, SVM's linearity limits its ability to model complex, non-linear relationships in the nutritional data. The slightly worse performance compared to Logistic Regression may come from its sensitivity to feature scaling or class imbalance, despite using balanced class weights.

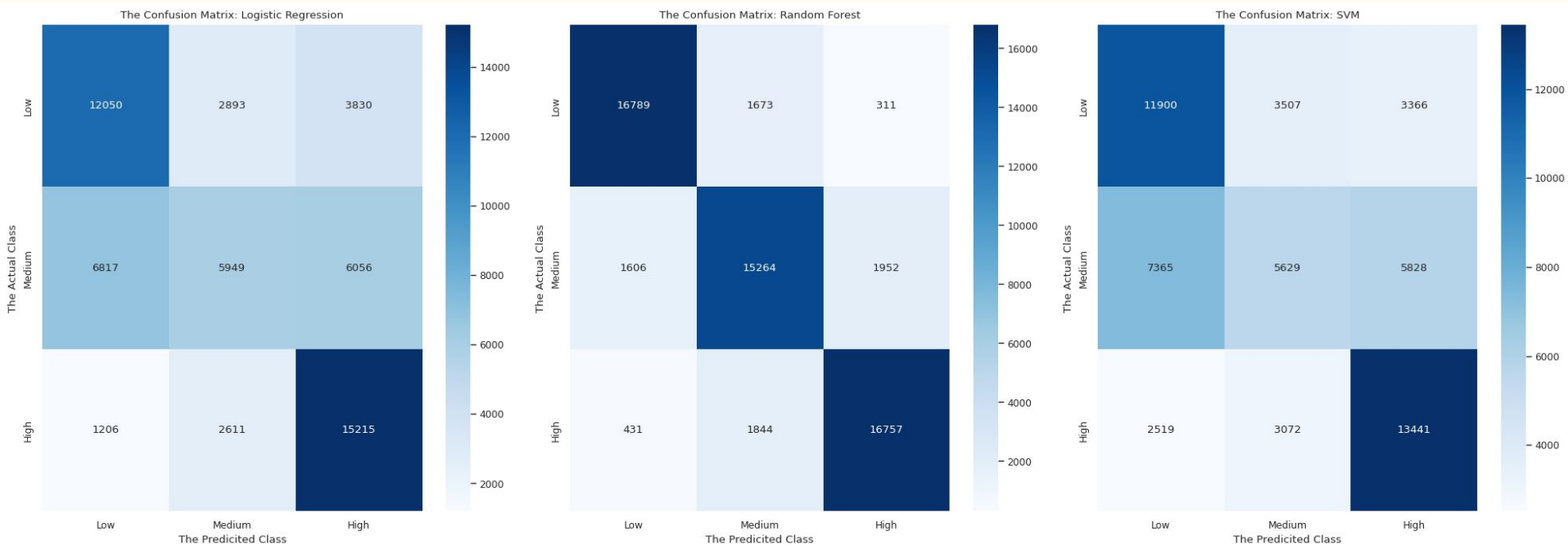
```
# Model #3 Linear SVM
svm_model = LinearSVC (
    dual = 'auto',
    random_state = seed,
    class_weight = 'balanced'
)
```

Which model performed the best?

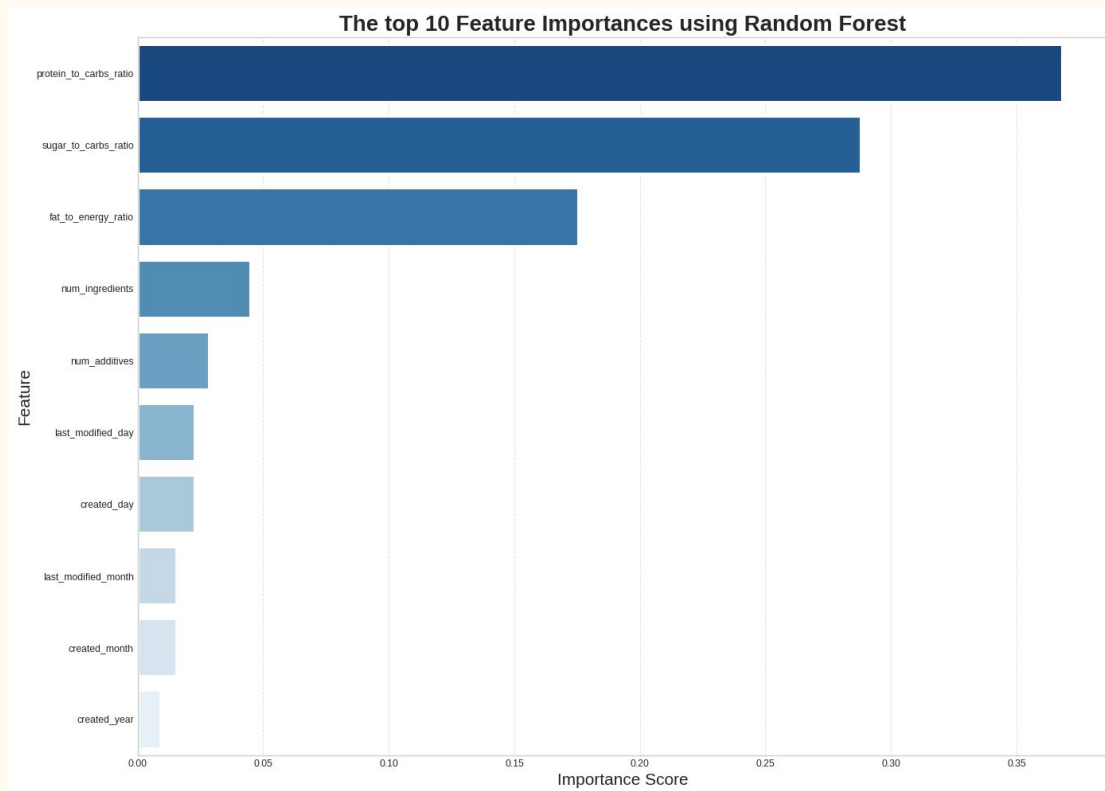


The Random Forest Model had the highest accuracy and precision with 86% accuracy.

Comparing the Models with a Confusion Matrix



Feature Importance using Random Forest



Optimization/Tuning, Interpretation, Reporting

GridSearchCV Hyperparameter Tuning

GridSearchCV is an effective hyperparameter tuning method in Scikit-learn that exhaustively searches through all possible combinations, or cross-validation, of parameters provided.

Improvements: The 0.35% improvement over the untuned model (86.20%) indicates that GridSearchCV identified better hyperparameters, enhancing generalization.

Drawbacks: The limitations of Google Colab does not allow for more extensive parameters which may result in more accurate results.

```
rf = RandomForestClassifier(class_weight='balanced', random_state=seed)
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    verbose=1
)
start_time = time.time()
grid_search.fit(X_trained_scaled, y_train)
tuning_time = time.time() - start_time
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_
best_score = grid_search.best_score_
```

```
# Defines the parameter grid

param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [10, 20, None],
    'max_features': ['sqrt']
}
```

Tuning and Optimization Results

Fitting 3 folds for each of 6 candidates, totalling 18 fits

Best Parameters: {'max_depth': None, 'max_features': 'sqrt', 'n_estimators': 100}

Best Cross-Validation Accuracy: 0.8609

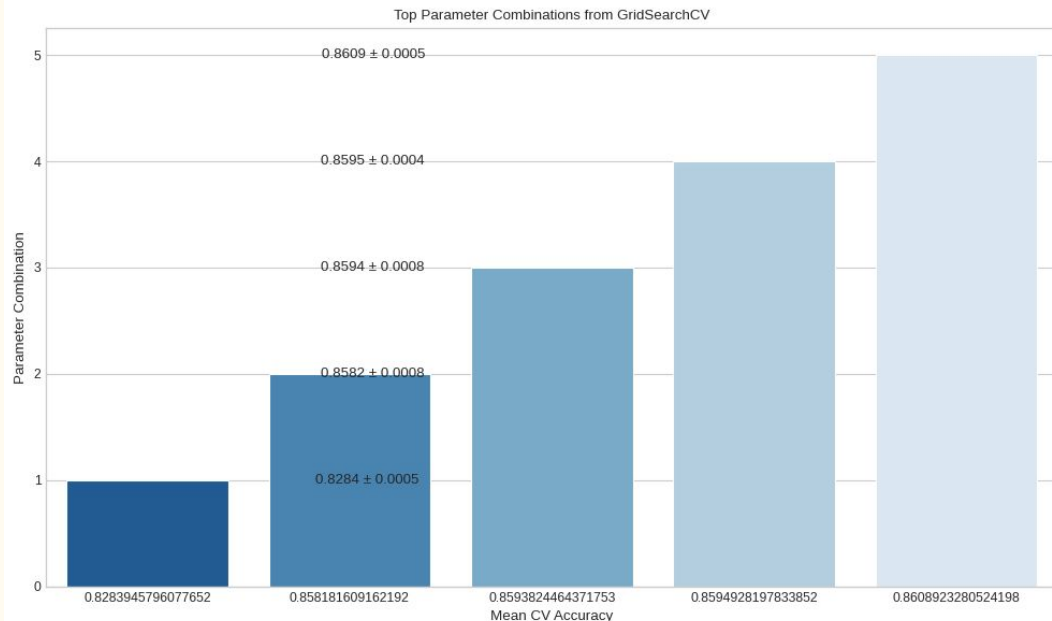
Tuning Time: 425.65 seconds

Test Accuracy: 0.8654

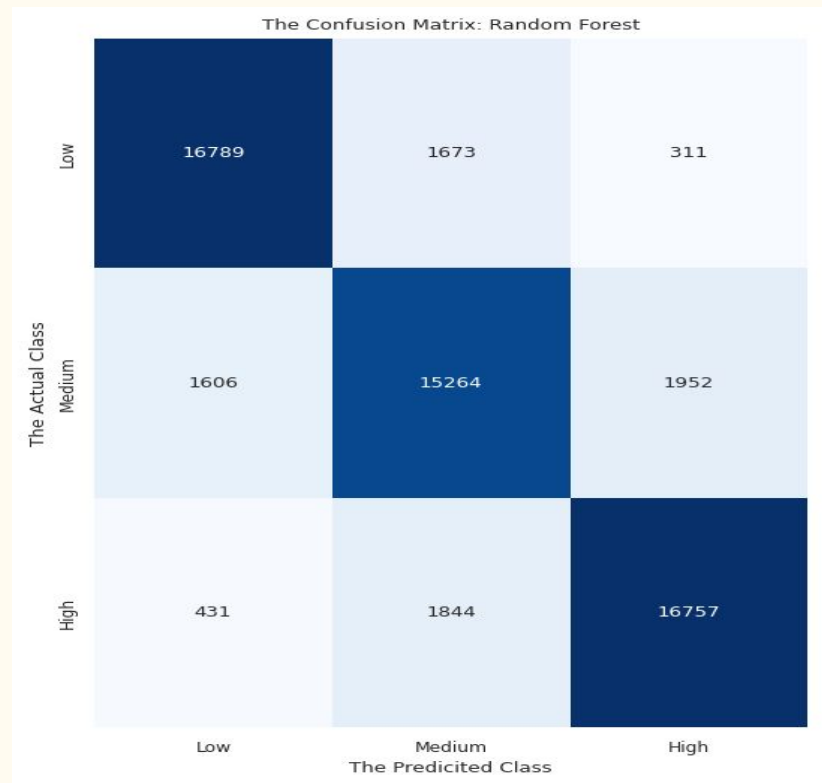
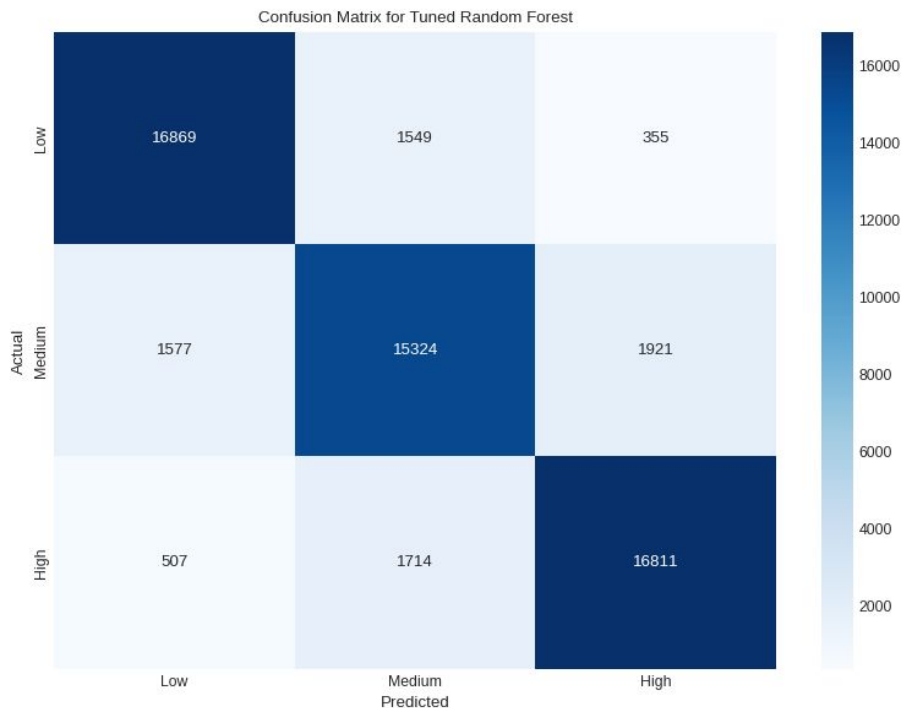
Test Precision: 0.8651

Test Recall: 0.8654

Test F1-Score: 0.8652



Comparing the Results



Performance Analysis

Random Forest outperformed both Logistic Regression and SVM, with about 30% higher accuracy. This suggests it generalizes better to the test set, likely due to its ability to model complex feature interactions.

Precision, Recall, and F1-Score: Random Forest metrics shows consistent performance across all health score classes. Logistic Regression and SVM show lower and less balanced metrics, suggesting they struggle with certain classes.

Computational Efficiency: Random Forest requires more computational resources compared to Logistic Regression and SVM.

Dataset Fit: The dataset's nutritional features, including ratios and features, likely contain non-linear patterns that Random Forest can effectively capture. The poor performance of linear models suggests that the data is not linearly separable, showing the importance for non-linear modeling approaches.

Challenges/Obstacles

- Some of the challenges we incurred included dealing with missing values, which required preprocessing. We sifted through the data, ensuring we left the relevant and meaningful columns and incorporated that data into our project.
- Another challenge we faced was in deciding how to create a meaningful and insightful health score, which we used to establish a classification on low/ medium, or highly healthy foods. There were many factors at our disposal to determine the score like specific vitamins, fats, etc, and the ones which we chose directly impact the results.
- Our dataset had major class imbalances, which we addressed by dividing the `health_score_scaled` column into three quantile-based bins, which separated them into a low, medium and high health score to balance out these classes.



Conclusion

Our machine learning pipeline classifies food products into Low, Medium, and High health score classes using the Open Food Facts dataset (~356,027 products).

Through comprehensive data preprocessing, feature engineering, exploratory data analysis, model development, and hyperparameter tuning, the project delivered a high-performing Random Forest Classifier, achieving a test accuracy of 86.54% (Precision: 0.8651, Recall: 0.8654, F1-Score: 0.8652) after GridSearchCV optimization.

Next Steps & Future Directions

—

Next Steps

Feature Expansion: Integrate additional datasets or features

Real-Time Processing: Optimize Random Forest for faster inference in real-time applications

Expand Dataset: We would like to bring in another dataset that includes carbon-footprint data linked to food items for a more comprehensive study

Practical Implications

Consumer Applications: The model can power apps for real-time food classification, aiding healthier dietary choices.

Industry Impact: Manufacturers can reformulate products to reduce unhealthy nutrients, meeting consumer/regulatory demands.

Public Health: Supports dietary guidelines, reducing diet-related diseases like obesity and diabetes.

Thank you!